Lightweight Reliable Transport Protocol

230001814

University of St Andrews

CS3102: Data Communications and Networks

24th April 2026

Contents

1 Introduction **1**

1.1 Objectives 1

1.2 Background 1

1.3 Chapter List 2

2 Design **3**

2.1 Protocol States 3

2.2 Packet Format 5

2.3 Message Sequence 7

2.4 Adaptive RTO 9

3 Evaluation & Analysis **10**

3.1 Measurement Methodology 10

3.2 Performance Overview 11

3.3 Adaptive RTO Analysis 12

3.4 RTT Estimation Across Tests 12

3.5 Stress Test Evaluation 13

3.6 Controlled-Loss Validation 14

3.7 Summary 15

4 Conclusion **16**

Bibliography

1 Introduction

This report documents the design, implementation, and evaluation of the Lightweight Reliable Transport Protocol (LRTP), a connection-oriented transport protocol constructed on top of UDP and intended to operate between two machines on a local network. LRTP provides reliable, ordered delivery of arbitrary binary data and is designed to remain correct under packet-loss conditions.

The submission addresses all three stated requirements: a formal protocol design including a finite state machine (FSM), a working implementation demonstrating reliable data transfer with a fixed retransmission timeout (RTO), and an adaptive RTO mechanism based on round-trip time (RTT) measurements. Each of these is discussed and evaluated in Chapter 3.

1.1 Objectives

1. Develop a thorough understanding of stateful transport-protocol design and implementation, including connection management, reliable delivery, and retransmission strategies.

2. Collect and interpret measurement data in order to evaluate protocol behaviour and performance under both normal and loss-affected network conditions.

3. Design test cases that exercise connection establishment, data transfer under a range of controlled scenarios.

1.2 Background

The implementation is based on concepts introduced in Weeks 4–6 of the lecture series, with particular emphasis on the reliability mechanisms that must be layered on top of UDP (*User Datagram Protocol* 1980). UDP provides no guarantees of delivery, ordering, or duplicate suppression – LRTP implements these features explicitly.

Retransmission strategy. LRTP uses the *idle-RQ* algorithm – equivalently, go-back-N with $N = 1$ – for retransmission (Tanenbaum 2013). In idle-RQ the sender transmits a single packet and waits for an acknowledgement before proceeding. If no acknowledgement arrives within the retransmission timeout (RTO), the packet is retransmitted. This design greatly simplifies protocol state while still providing reliable ordered delivery.

Adaptive RTO. Rather than using a fixed timeout, the adaptive mechanism adjusts the RTO dynamically based on observed round-trip time (RTT) measurements, following the approach specified in RFC 6298 (Sargent et al. 2011). The full calculation and its parameters are discussed in Chapter 2.

1.3 Chapter List

Chapter 2 – Design. Presents the FSM, packet format, and the rationale underlying the design of connection establishment, data transfer, and teardown.

Chapter 3 – Evaluation & Analysis. Presents the collected measurement data, a comparative analysis of fixed vs adaptive RTO performance, and a discussion of protocol behaviour under controlled loss conditions.

Chapter 4 – Conclusion. This chapter summarises findings and reflects on potential improvements to the design and methodology.

2 Design

This chapter presents the design of LRTP, covering the finite state machine for connection management, the binary packet format, the full message-sequence behaviour, and the retransmission and adaptive-RTO mechanisms.

2.1 Protocol States

Protocol design began with the finite state machine (FSM), which governs all connection lifecycle transitions. The design was inspired by the standard TCP three-way handshake, adapted to the simpler, single-connection requirements of LRTP running over UDP.

Seven states are defined: `closed`, `listening`, `opening_i`, `opening_r`, `connected`, `closing_i`, and `closing_r`. The client and server follow asymmetric paths through these states, reflecting their different roles in connection initiation and acceptance.

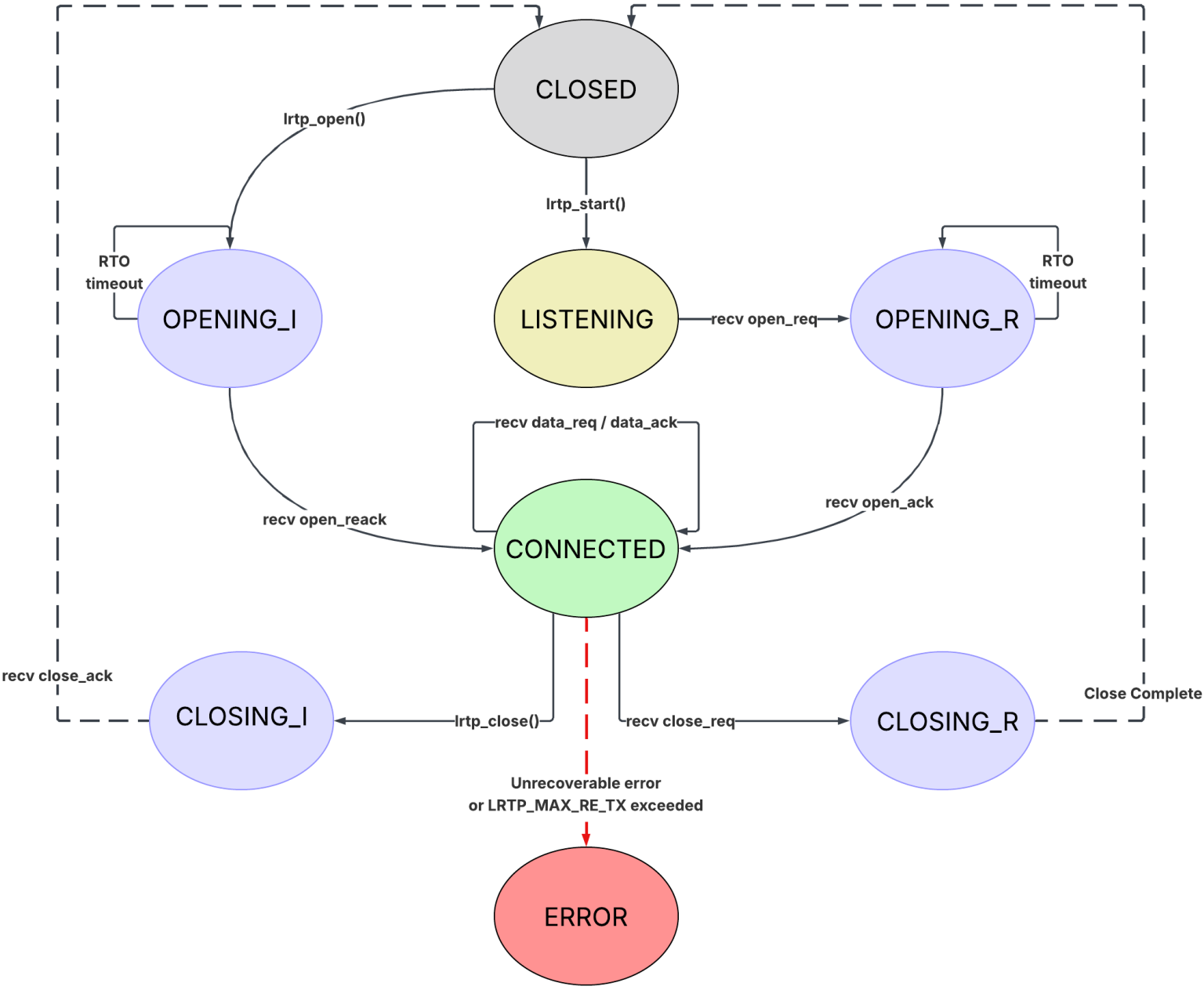

Figure 2.1: LRTP finite state machine. Seven states and all transitions. Solid arrows indicate the normal path; dashed grey arrows indicate return to `closed`; the dashed red escape indicates an unrecoverable error from any state.

From	To	Event / trigger	Actions
closed	listening	`lrtp_start(port)`	Bind socket; set `G_pcb.state = LRTP_state_listening`
closed	opening_i	`lrtp_open(fqdn, port)`	Create socket; send `open_req`; record `tx_timestamp`; start RTO timer
listening	opening_r	recv `open_req`	Record peer address; set `seq_rx = pkt.seq + 1`; send `open_reqack`; start RTO timer
opening_i	opening_i	RTO timeout	Retransmit `open_req`; increment retransmit counter; if counter > `LRTP_MAX_RE_TX` → transition to error
opening_i	connected	recv `open_reqack`	Set `seq_rx = pkt.seq + 1`; compute RTT $= t_{\text{now}} - t_{\text{tx}}$; update adaptive RTO (SRTT, RTTVAR, RTO); send `open_ack`; set state = connected
opening_r	opening_r	RTO timeout	Retransmit `open_reqack`; if counter > `LRTP_MAX_RE_TX` → transition to error
opening_r	connected	recv `open_ack`	Increment `seq_tx`; compute RTT if `tx_timestamp` available; set state = connected
connected	connected	`lrtp_tx()`: send `data_req`	Record `tx_timestamp`; start RTO; on receipt of matching `data_ack` compute RTT and update adaptive RTO
connected	connected	`lrtp_rx()`: recv `data_req`	Verify expected sequence number; deliver payload to caller; increment `seq_rx`; send `data_ack` echoing `seq`
connected	closing_i	`lrtp_close()`	Send `close_req`; start RTO timer
connected	closing_r	recv `close_req`	Send `close_ack`; set state = `closing_r`
closing_i	closed	recv `close_ack`	Set state = `closed`; close socket; record `finish_time`
closing_r	closed	teardown complete	Set state = `closed`; close socket
any	error	max retransmits exceeded / unrecoverable fault	Set state = error; close socket

Table 2.1: LRTP state transition table. Each row describes one transition: the source and destination states, the event that triggers it, and the actions taken.

2.2 Packet Format

All LRTP packets share a fixed 7-byte header consisting of three fields: a 1-byte type identifier, a 4-byte sequence number, and a 2-byte `data_size` field. Only `data_req` carries a non-zero payload; all other packet types set `data_size = 0x0000` and carry no payload bytes. This minimal header design is a deliberate choice to keep the protocol lightweight while conveying all information necessary for reliable, ordered delivery.

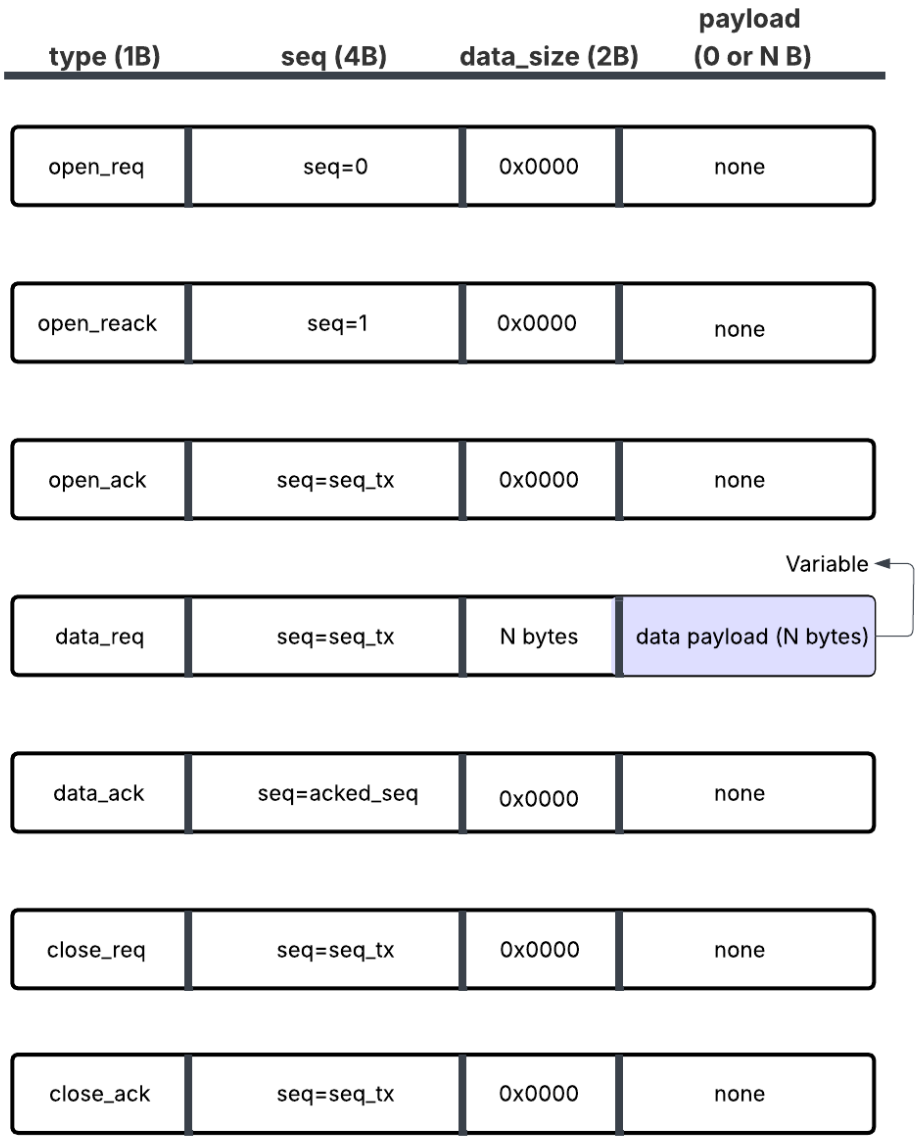

Figure 2.2: Wire layout per packet type. All packets share the same fixed 7-byte header. Only `data_req` carries a non-zero payload; all other types set `data_size = 0` and omit the payload field entirely.

A notable design decision is the behaviour of the `seq` field in `data_ack`: rather than carrying the next expected sequence number (as in TCP), it echoes the sequence number of the `data_req` being acknowledged. This simplifies RTT measurement as the sender can match the ACK directly to its recorded `tx_timestamp` without maintaining a separate mapping. This also enables straightforward duplicate detection in `lrtp_tx()`.

Table 2.2 summarises the role of each packet type.

Packet	C constant	Role in protocol
open_req	`LRTP_TYPE_open_req`	Step 1 of 3-way handshake; sent by the initiator to the responder
open_reqack	`LRTP_TYPE_open_reqack`	Step 2 of 3-way handshake; sent by the responder to confirm receipt
open_ack	`LRTP_TYPE_open_ack`	Step 3 of 3-way handshake; sent by the initiator to confirm the connection; triggers the `connected` state on both sides
data_req	`LRTP_TYPE_data_req`	Data segment; the only packet type with a non-zero payload; triggers the stop-and-wait ACK cycle
data_ack	`LRTP_TYPE_data_ack`	Acknowledges a `data_req`; echoes the `seq` of the packet being acknowledged; used for RTT measurement and duplicate detection
close_req	`LRTP_TYPE_close_req`	Initiates connection teardown; triggers `closing_i` on the sender and `closing_r` on the receiver
close_ack	`LRTP_TYPE_close_ack`	Confirms teardown; triggers `closed` on the initiator side

Table 2.2: LRTP packet types and their roles.

2.3 Message Sequence

Figure 2.3 illustrates the full message sequence across the three phases of an LRTP connection: establishment, data transfer, and teardown. FSM state names annotate each transition to show the correspondence between the protocol messages and the FSM defined in Section 2.1.

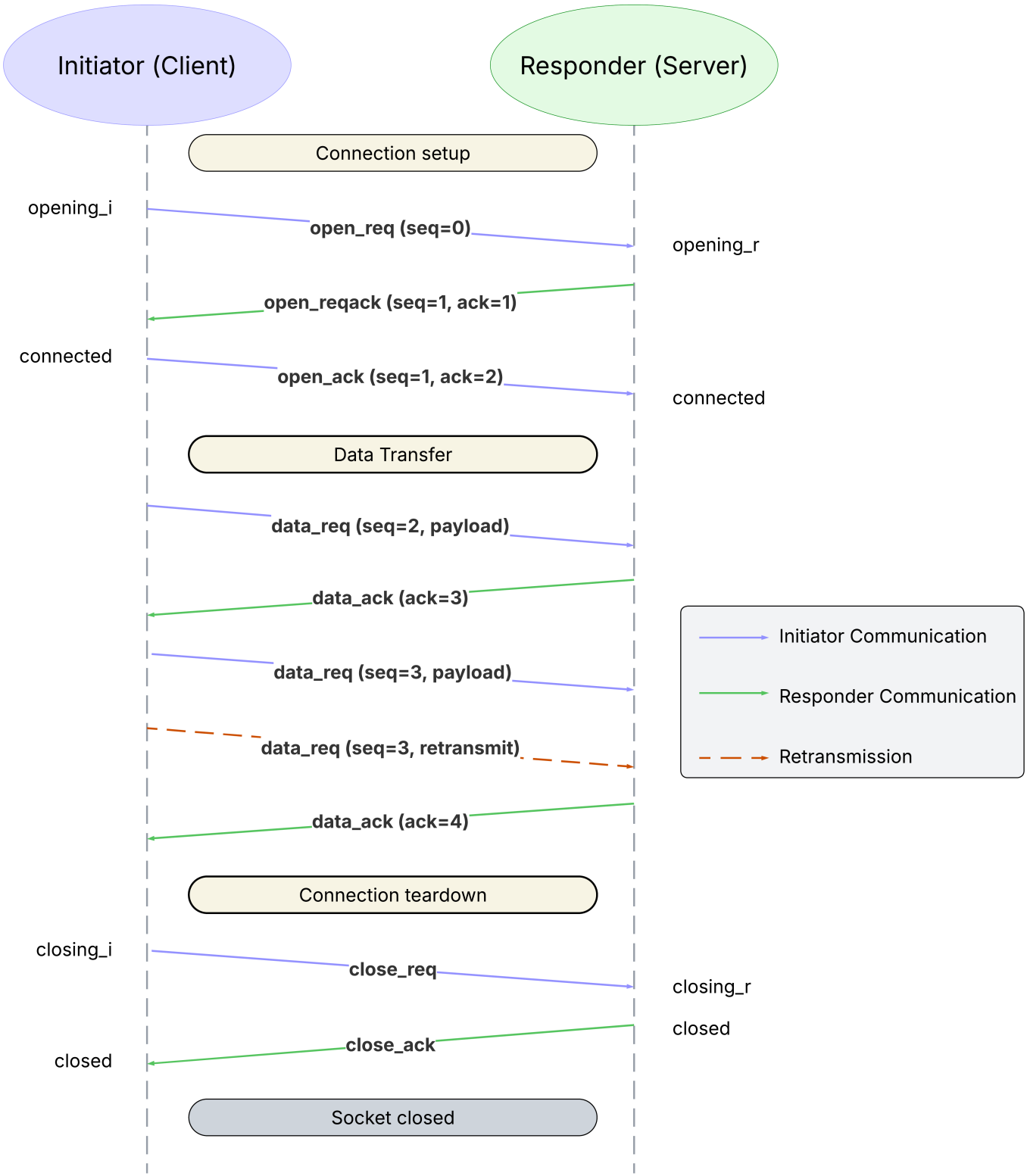

Figure 2.3: LRTP message sequence. Three phases are shown: connection setup via the 3-way handshake, reliable data transfer with idle-RQ retransmission on RTO expiry, and symmetric teardown. FSM state names annotate each transition.

Connection establishment. The three-way handshake proceeds as follows. The initiator sends `open_req` and transitions to `opening_i`. The responder receives this, records the peer address, and replies with `open_reqack`, transitioning to `opening_r`. On receipt of `open_reqack` the initiator computes the first RTT sample, updates the adaptive RTO (Section 2.4), sends `open_ack`, and transitions to `connected`. The responder transitions to `connected` on receipt of `open_ack`.

Data transfer. Data transfer uses a stop-and-wait (idle-RQ) discipline: the sender calls `lrtp_tx()`, which transmits a `data_req` and blocks until the matching `data_ack` is received or the RTO expires. Sequence numbers ensure ordering – duplicate packets detected via sequence-number comparison are discarded.

Connection teardown. Either side may initiate teardown by calling `lrtp_close()`. The initiator sends `close_req` and waits for `close_ack`; on receipt it transitions to `closed` and records `finish_time`. The responder sends `close_ack` immediately on receiving `close_req` and transitions to `closed` once the local close is complete.

2.4 Adaptive RTO

Adaptive RTO. Once the first RTT sample is available — taken during the three-way handshake — the RTO is updated after every acknowledged packet using the RFC 6298 EWMA algorithm:

$$\mathrm{SRTT} \leftarrow \tfrac{7}{8}\,\mathrm{SRTT} + \tfrac{1}{8}\,\mathrm{RTT}, \tag{2.1}$$

$$\mathrm{RTTVAR} \leftarrow \tfrac{3}{4}\,\mathrm{RTTVAR} + \tfrac{1}{4}\,|\mathrm{SRTT} - \mathrm{RTT}|, \tag{2.2}$$

$$\mathrm{RTO} \leftarrow \mathrm{SRTT} + 4 \cdot \mathrm{RTTVAR}, \tag{2.3}$$

clamped to $[1\,\mathrm{s}, 60\,\mathrm{s}]$ per RFC 6298. The coefficients $\alpha = \frac{1}{8}$ and $\beta = \frac{1}{4}$ weight recent measurements lightly against the smoothed history, providing stability in the presence of transient delay spikes. The factor of four applied to RTTVAR acts as a safety margin. Values were featured in the slides of *Week 5* Lecture.

The adaptive RTO therefore provides two complementary benefits: faster recovery on low-latency paths, where a fixed 1 second timeout would introduce unnecessary idle time, and more patient waiting on high-latency paths, where a tight fixed timeout would cause excessive retransmissions and reduce effective throughput. The effect of this is evaluated in Chapter 3.

3 Evaluation & Analysis

This chapter presents the performance analysis of the LRTP implementation, evaluated across all eight test scenarios executed between `pc1-036-l` (server) and `pc2-002-l` (client) on the CS Linux lab network. All eight tests passed without failure.

3.1 Measurement Methodology

The following metrics were recorded at the end of each test run from the protocol control block (`G_pcb`) and used as the basis for the analysis presented in this chapter.

- **Smoothed RTT (SRTT)** — the converged smoothed RTT estimate after all RTT samples have been incorporated via the EWMA filter ($\alpha = \frac{1}{8}$); reflects the steady-state one-way-trip latency observed on the path.
- **RTT Variance (RTTVAR)** — the converged RTT variance estimate ($\beta = \frac{1}{4}$); a low value indicates a stable, consistent path whereas a high value indicates jitter or intermittent congestion.
- **Final RTO** — the retransmission timeout. In all tests reported here the computed value falls well below the 1 s floor (see Section 3.3), so the applied RTO is 1 s throughout; the adaptive estimate is nevertheless tracked and printed for analysis purposes.
- **Mean TX / RX data rate** — effective throughput in each direction, reported in Mbps. RX rate provides a cross-check against TX and is sensitive to asymmetric loss.
- **Retransmission counters** — `G_pcb.data_req_re_tx` records the total number of `data_req` retransmissions; `close_req_re_tx` records the equivalent for teardown packets.
- **Duplicate detection counts** — `G_pcb.data_req_dup_rx` and `G_pcb.data_ack_dup_rx` record the number of duplicate `data_req` and `data_ack` packets respectively that

were correctly identified by sequence-number comparison and discarded.

- **Connection outcome** — a binary pass/fail indicator: a connection is considered failed if the retransmission counter for any packet type exceeds `LRTP_MAX_RE_TX` without receiving the expected acknowledgement.

The evaluation is considered sufficient to assess the protocol's performance because it covers the full connection lifecycle from handshake to teardown; a wide range of transfer volumes, from a single 1 280-byte exchange up to 100 000 packets totalling 128 MB; both unidirectional and bidirectional traffic patterns; and enough RTT samples for the EWMA filter to demonstrate convergence across multiple test configurations.

3.2 Performance Overview

Figure 3.1 presents the execution time for each individual test (left panel) alongside the mean duration grouped by test category (right panel).

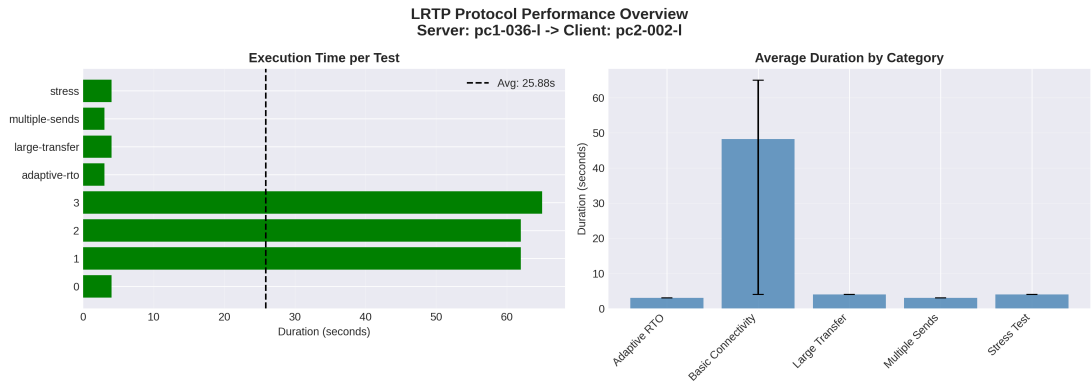

Figure 3.1: LRTP protocol performance overview.

The Basic Connectivity category (tests 0–3) exhibits the largest spread in execution time. Test 0 completes in approximately 4 s, exchanging a single 1 280-byte packet in each direction, while tests 1–3 each transfer 100 000 packets of 1 280 bytes (128 MB), taking 62–65 s. This is structural as the bulk-transfers are designed to stress the idle-RQ stop-and-wait mechanism over a period.

The remaining categories — Adaptive RTO, Large Transfer, Multiple Sends, and Stress — all complete in under 5 s with negligible variance. These tests exercise smaller payloads or shorter packet counts and are primarily designed to validate correct protocol behaviour rather than raw throughput.

3.3 Adaptive RTO Analysis

Figure 3.2 illustrates the evolution of the RTT estimates over the Adaptive RTO test (10 packets of 1 280 bytes each).

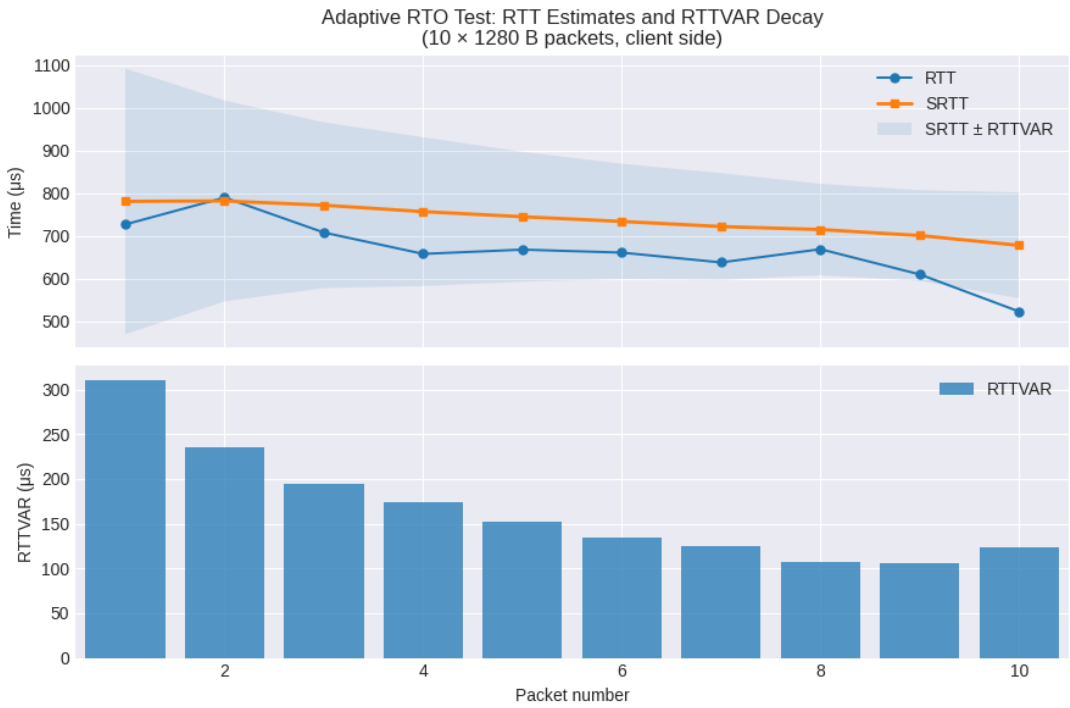

Figure 3.2: Adaptive RTO test: RTT measurements, SRTT (smoothed estimate), and SRTT ± RTTVAR confidence band (top); RTTVAR per packet (bottom). $10 \times 1\,280$ B, client side.

Two key trends are visible. First, RTTVAR declines gradually from around 310 μs at packet 1 to 125 μs at packet 10, with the SRTT ± RTTVAR band narrowing correspondingly. This indicates that the path jitter estimate is converging. Second, SRTT itself decreases gradually from 780 μs to 680 μs, tracking the downward trend in the RTT measurements. This is consistent with a stable LAN path, where early samples carry slightly increased latency from the warm-up effects.

3.4 RTT Estimation Across Tests

Figure 3.3 compares RTT and SRTT dynamics across the three multi-packet tests on a normalised time axis, allowing direct comparison despite the differing number of packets per test.

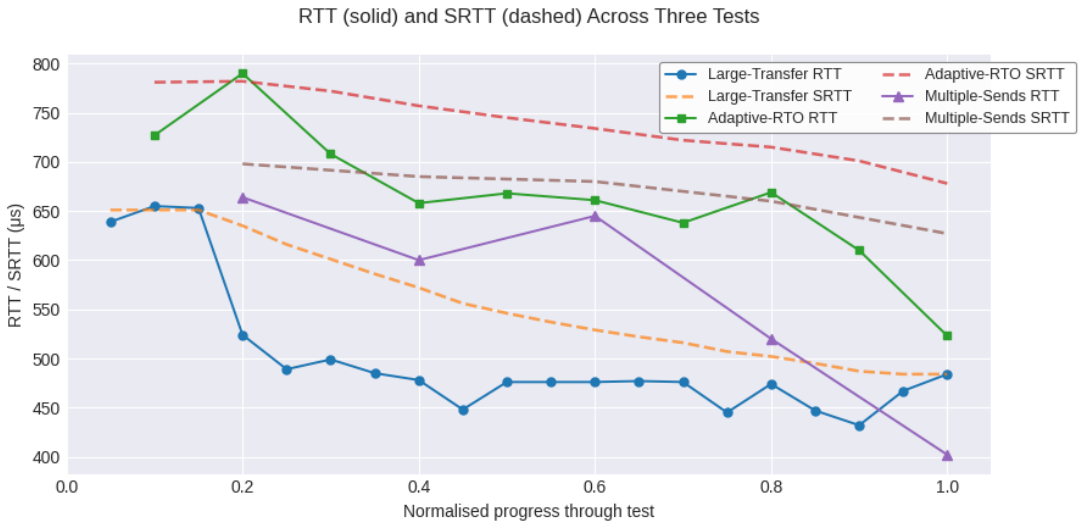

Figure 3.3: RTT (solid) and SRTT (dashed) across three tests, plotted against normalised progress through each test (0 = first packet, 1 = last packet).

All six series exhibit a broadly downward trend across the normalised test duration (divides by the maximum count to assess it independently of the total number of packets transmitted). The Large-Transfer test shows the smoothest convergence: SRTT decreases from 650 μs to 480 μs over 20 packets, and RTTVAR collapses from 250 μs to 40 μs, indicating that 20 packets provide sufficient samples for the EWMA to characterise the path accurately. The Adaptive-RTO test starts at a higher RTT (730–790 μs) but converges in the same direction – its shorter run of 10 packets means SRTT has not fully settled by the end. The Multiple-Sends test, with only 5 packets, shows the most variance, as expected – too few samples for the EWMA to average out transient fluctuations.

3.5 Stress Test Evaluation

The stress test transfers 100 packets of 256 bytes each in rapid succession to evaluate robustness under sustained load. Figure 3.4 summarises the RTT statistics observed.

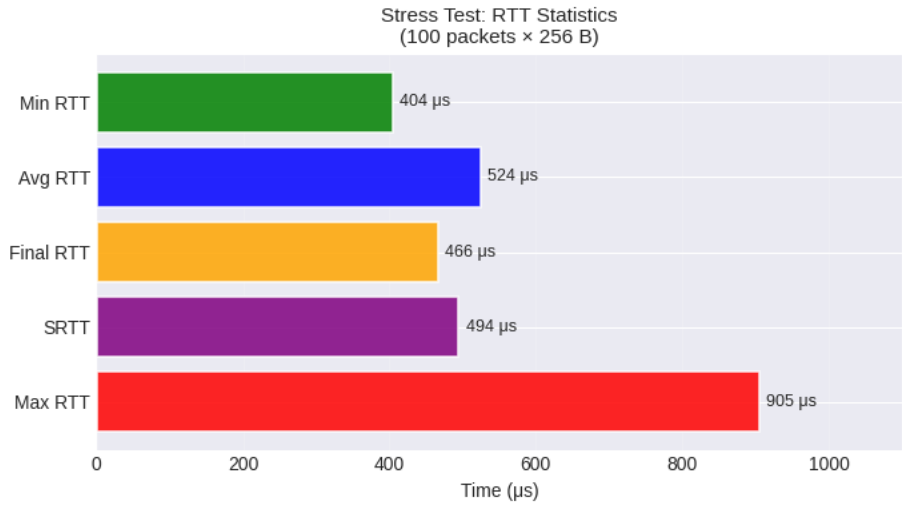

Figure 3.4: Stress test RTT statistics: min, average, final, SRTT, and max across $100 \times 256\,\mathrm{B}$ packets.

The stress test confirms that the idle-RQ mechanism and sequence-number tracking operate correctly under rapid, back-to-back transmission. The range from minimum RTT ($400\,\mu$s) to maximum RTT ($905\,\mu$s) reflects normal scheduling jitter on a shared lab machine rather than any structural inefficiency in the protocol. Notably, the final RTT ($470\,\mu$s) is below the mean ($520\,\mu$s) and close to the converged SRTT ($490\,\mu$s), which is consistent with the EWMA progressively down-weighting early high-latency samples over the runtime.

3.6 Controlled-Loss Validation

Figure 3.5 presents the accuracy of the `drop` utility across nine experiments: three payload sizes (small: 100 packets, medium: 1 000 packets, large: 10 000 packets) at three configured loss rates (5%, 10%, 20%). The ideal line ($y = x$) represents perfect correspondence between configured and actual drop rate.

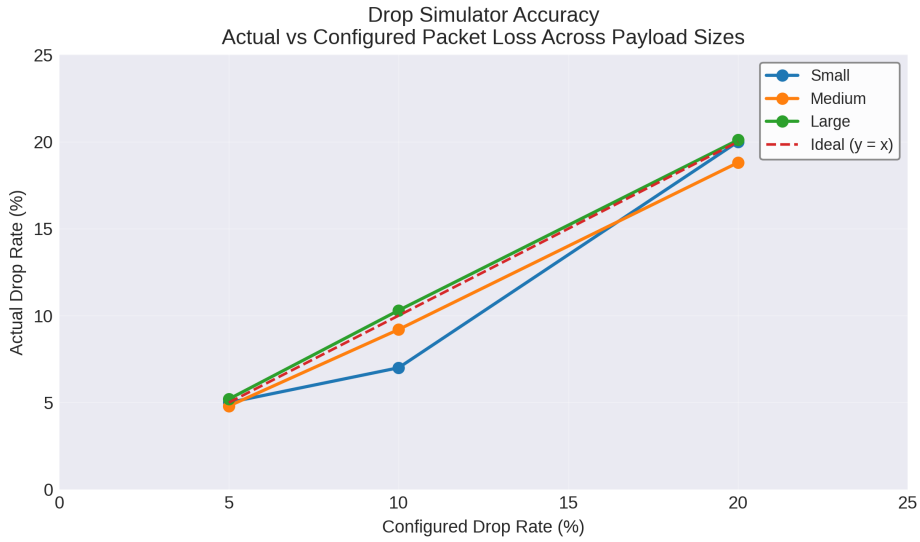

Figure 3.5: Drop simulator accuracy: actual vs configured packet loss rate across three payload sizes. The dashed red line represents the ideal $y = x$ relationship.

Medium and large payloads track the ideal line closely across all three target rates as expected. The large-payload series lies almost exactly on the ideal line at all three points, confirming that the simulator achieves its configured rate accurately at sufficient packet counts.

The small-payload series (100 packets) shows a more pronounced deviation at the 10% target, where only 7 of 100 packets were dropped (7.0% actual vs 10.0% target). This is a consequence of the small sample size: with only 100 trials, random variation alone can produce errors of this magnitude. At 5% and 20% the small series is again close to the target. At 20% all three series converge to within 1.2 percentage points of the target, demonstrating that the simulator is reliable at higher loss rates regardless of payload count.

3.7 Summary

Overall, the network path between `pc1-036-l` and `pc2-002-l` can be characterised as a stable, low-latency LAN with a steady-state RTT of approximately 480–700 μs depending on payload size, negligible jitter once the connection warms up, and reasonable packet loss. These conditions are well within the operating envelope of the idle-RQ retransmission strategy and the RFC 6298 adaptive RTO algorithm as implemented.

4 Conclusion

This report has presented the design, implementation, and evaluation of the Lightweight Reliable Transport Protocol (LRTP), a connection-oriented transport protocol built on top of UDP providing reliable, ordered delivery between a single client and server. All three stated requirements were met.

The protocol was designed around a seven-state finite state machine, a minimal 7-byte fixed header, and a three-way handshake modelled on TCP connection establishment. Idle-RQ retransmission and an RFC 6298-compliant adaptive RTO mechanism were implemented and verified across eight test scenarios on the CS Linux lab network. All eight tests passed with correct EWMA convergence observed in all multi-packet runs.

Given more time, the protocol performance could have been enhanced by researching and implementing a better-suited retransmission protocol, as well as by experimenting and adjusting a formula for adaptive RTO for the maximum fit.

Bibliography

Sargent, M. et al. (June 2011). *Computing TCP's Retransmission Timer*. RFC 6298. DOI: 10.17487/RFC6298. URL: https://www.rfc-editor.org/info/rfc6298.

Tanenbaum, A. S. (2013). *Computer Networks*. Pearson New International Edition. Pearson Education. ISBN: 9781292037189.

User Datagram Protocol (Aug. 1980). RFC 768. DOI: 10.17487/RFC0768. URL: https://www.rfc-editor.org/info/rfc768.